Estrategia Definitiva de Solución

Agente QA — Análisis de 5 Capas

CRÍTICO · PIPELINE V2 · INTRALE PLATFORM

Generado: 9 de abril de 2026 · Commander Pipeline V2

📋 Resumen Ejecutivo

El agente de QA es el **único agente del pipeline que nunca completó exitosamente** de forma consistente. De 10 ejecuciones documentadas, solo 1 fue exitosa (issue #2019 — delivery login). Esto bloquea todo el flujo de merge a `main` ya que el gate de QA es obligatorio.

Tras analizar los 10 reportes de rechazo, logs del pipeline, git history y las 16 issues con `qa:failed`, identificamos **5 causas raíz que se potencian entre sí**, formando un ciclo vicioso que impide el avance del pipeline.

10%	16	1
Tasa de éxito QA	Issues bloqueados (qa:failed)	Caso exitoso (#2019)

📊 Diagnóstico — Datos Duros

Causa de Rechazo	Frecuencia	Porcentaje	Impacto
Emulador Android no disponible	6 de 10	60%	Bloqueo total — sin emulador no hay test posible
Video inválido (<200KB) o sin audio	3 de 10	30%	Evidencia incompleta — rechazo por gate de evidencia
Backend remoto no responde	1 de 10	10%	Timeout en health-check del Lambda AWS

El caso exitoso: Issue #2019

El **único éxito registrado** fue el issue #2019 (delivery login). Ese día: el emulador estaba corriendo, grabó 45s de video con audio narrado, y pasó todos los gates. Esto prueba que el sistema **puede funcionar** cuando la infraestructura está disponible.

🔄 El Ciclo Vicioso

Las 5 causas raíz no son independientes — se potencian formando un loop destructivo:

```
Dev entrega rama vacía/incompleta
↓
QA se lanza sin emulador ni APK disponible
↓
Falla por infraestructura → etiqueta qa:failed
↓
Circuit breaker aplica cooldown (3 intentos máx)
↓
Pipeline re-lanza → mismo resultado
↓
16 issues acumulados sin poder mergear
↻ se repite
```

🛡️ Estrategia de Solución — 5 Capas

Cada capa es independiente y resuelve una causa raíz específica. Se implementan en orden de impacto.

1 Emulador Persistente como Servicio

Resuelve el 60% de los fallos de forma inmediata

Problema

El script `qa-android.sh` levanta el emulador, pero cuando el agente QA termina o muere, el proceso del emulador muere con él. No hay watchdog ni tarea programada que lo mantenga vivo. El timeout de boot (180s) causa `exit 1` sin reintentos.

Solución

- Crear una **Windows Scheduled Task** que lance `virtualAndroid` al inicio del sistema y lo mantenga vivo
- Health-check cada 60 segundos via `adb devices` — auto-restart si está muerto
- **Desacoplado del ciclo de vida del agente QA** — el emulador siempre está disponible
- Con snapshot `qa-ready`, el boot es de ~40 segundos

Implementación técnica

- Script: `.pipeline/emulator-watchdog.js` (Node.js, corre como servicio)
- Registro: `schtasks /create /tn "IntraleEmulator" /tr "node emulator-watchdog.js" /sc onstart`
- El watchdog escribe estado en `.pipeline/emulator-status.json`
- Pulpo lee ese archivo antes de lanzar QA

2 Pre-Flight Checks en Pulpo

Evita desperdiciar tokens en ejecuciones condenadas al fracaso

Problema

El pipeline lanza el agente QA sin verificar que la infraestructura necesaria esté disponible. El agente arranca, descubre que no hay emulador, intenta levantarlo, falla, y consume tokens durante 10-15 minutos antes de reportar `qa:failed`.

Solución

Agregar 3 verificaciones obligatorias en `pulpo.js` **antes** de lanzar el agente QA:

- **¿Emulador vivo?** → `adb devices | grep emulator`
- **¿APK disponible?** → Verificar existencia en `qa/artifacts/`
- **¿Backend responde?** → POST a `/intrale/signin` con body vacío → esperar HTTP 400

Acciones automáticas

- Si falta algo → **reparar automáticamente antes de lanzar**, no después
- Si no se puede reparar → marcar issue como `blocked:infra` en lugar de `qa:failed`
- Evita que el circuit breaker penalice por fallos de infraestructura

3 Separar QA-API de QA-Android

Desbloquea issues de backend que no necesitan emulador

Problema

Issues puramente de backend (API endpoints, funciones Lambda) pasan por el mismo flujo que requiere emulador Android. Quedan bloqueados innecesariamente esperando un emulador que no necesitan.

Solución

Labels del Issue	Modo QA	Requiere Emulador
`area:backend` sin `app:*`	QA-API (HTTP requests + validación de respuestas)	No
`app:client`, `app:business`, `app:delivery`	QA-Android (Maestro flows + video)	Sí
`area:infra` sin `app:*`	`qa:skipped` con justificación	No

Implementación

- Pulpo determina el modo QA según los labels del issue
- Crear script `qa/scripts/qa-api.sh` para testing HTTP puro
- QA-API genera evidencia con logs de requests/responses (no necesita video)

4 Build Produce APK como Artefacto

Elimina la re-compilación dentro del agente QA

Problema

El skill de QA espera encontrar el APK pre-compilado en `qa/artifacts/`, pero el directorio no existe. La fase de build nunca copia el APK allí. QA tiene que compilar desde cero: 5+ minutos, riesgo de OOM, fallos frecuentes.

Solución

- Después de un build exitoso del agente Dev, copiar APK a `qa/artifacts/composeApp-{flavor}-debug.apk`
- QA solo instala y testea — **nunca compila**
- Reduce tiempo de QA de ~25 min a ~15 min
- Elimina OOM por builds concurrentes

Implementación

- Hook post-build en el agente Dev: copiar APK al directorio de artefactos
- Pre-flight check (Capa 2) valida que el APK existe y es reciente (<24h)
- Si el APK es viejo o falta → rebuild automático antes de lanzar QA

5 Evidencia con Degradación Gradual

Permite aprobación con evidencia parcial en vez de rechazo total

Problema

El gate de evidencia es all-or-nothing: requiere video >200KB **Y** audio narrado. Si el emulador falla, no hay video, no hay audio, rechazo completo. No hay fallback.

Solución — Cadena de fallback

Nivel	Tipo de Evidencia	Resultado
🥇 Ideal	Video >200KB + audio narrado (edge-tts)	qa:passed
🥈 Aceptable	Screenshots secuenciales + logs de API	qa:passed (con nota)
🥉 Mínimo	Solo logs de API (backend issues)	qa:passed (solo para `area:backend`)
⏸️ Bloqueado	Infra no disponible, cambios verificables	qa:blocked-env (retry automático)
❌ Rechazo	Rama sin cambios de código / criterios inverificables	qa:failed

Validación pre-QA de rama

- Verificar que la rama tiene cambios reales de código Kotlin (no solo config/docs)
- Si la rama está sincronizada con main sin cambios → rechazar **antes** de lanzar QA
- Esto previene el caso del issue #2041 (zero cambios, QA lanzado innecesariamente)

🗺️ Roadmap de Implementación

FASE 1 — QUICK WINS

Capas 1 + 2

Emulador persistente + pre-flight checks. Resuelve el 60% de fallos. Esfuerzo: **Simple**.

FASE 2 — SEPARACIÓN

Capa 3

QA-API vs QA-Android. Desbloquea issues backend. Esfuerzo: **Medio**.

FASE 3 — OPTIMIZACIÓN

Capa 4

APK como artefacto de build. Reduce tiempo 40%. Esfuerzo: **Simple**.

FASE 4 — RESILIENCIA

Capa 5

Degradación gradual de evidencia. Elimina rechazos por infra. Esfuerzo: **Medio**.

Resultado esperado post-implementación

85%+	~15 min	0
Tasa de éxito QA proyectada	Tiempo promedio QA (hoy: 25+ min)	Tokens desperdiciados en infra

✅ Conclusión

El agente de QA no tiene un problema de lógica ni de código — tiene un problema de **infraestructura y resiliencia**. La prueba es que cuando la infra está disponible (caso #2019), funciona perfectamente.

La estrategia de 5 capas ataca cada punto de fallo de forma independiente. Las capas 1 y 2 son quick wins que resuelven el **70% del problema** con esfuerzo mínimo. Las capas 3-5 consolidan la solución para lograr un pipeline robusto y autónomo.

Con estas 5 capas implementadas, el agente de QA pasa de ser el cuello de botella del pipeline a ser un gate confiable y predecible.

Intrale Platform · Pipeline V2 Commander · Abril 2026